

Capitolo 4 - Thread e concorrenza

Status Done

Abbiamo detto che un processo è un programma in esecuzione con un unico percorso di controllo, tuttavia nei SO moderni un processo può avere più percorsi di controllo chiamati *thread*.

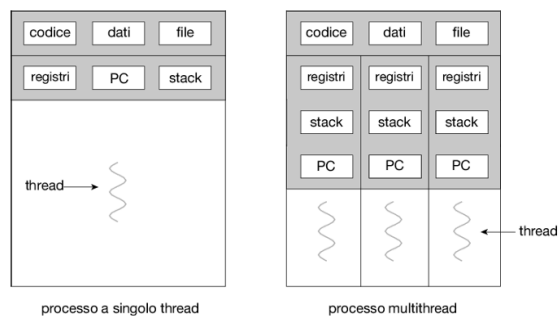
Un thread è l'unità di base d'uso della cpu e comprende:

- un identificatore di thread (id);
- un contatore di programma;
- un insieme di registri;
- una pila (stack).

Condivide con gli altri thread che appartengono allo stesso processo:

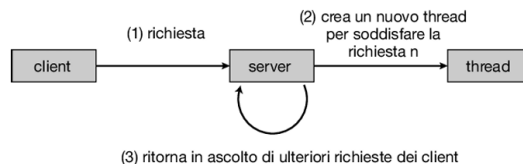
- la sezione del codice;
- la sezione dei dati;
- altre risorse di sistema.

Un processo tradizionale (*heavyweight process*), è composto da un solo thread. Un processo multithread è in grado di svolgere più compiti in modo concorrente.



Le moderne applicazioni sono in gran parte multithread, ovvero un singolo processo è composto da più thread di controllo che possono eseguire compiti diversi simultaneamente.

- **Scopo:** il multithreading consente alle applicazioni di gestire compiti multipli e complessi, come in un browser web (un thread per la grafica e uno per il download) o in word processor (un thread per l'input e uno per la correzione in background).
- **Sfruttamento multicore:** le applicazioni multithread sono progettate per sfruttare i sistemi **multicore**, eseguendo attività intensive in parallelo su diversi core e garantendo la scalabilità.
- **Esempio Server Web:** in sistemi ad alto traffico (come un server web), la gestione concorrente di molte richieste richiede l'uso di thread. Creare un nuovo processo per ogni richiesta è molto oneroso in termini di tempo e risorse. È più efficiente far sì che il server, in quanto processo multithread, crei un nuovo thread per servire ciascun client



- **Kernel SO:** anche la maggior parte dei kernel dei SO (es. Linux) è multithread, con thread dedicati a funzioni specifiche come la gestione dei dispositivi o della memoria.

Vantaggi

I benefici della programmazione multithread si classificano in quattro categorie principali:

1. **Tempo di risposta:** un'applicazione interattiva rimane attiva e reattiva all'utente anche se un thread è bloccato o sta eseguendo un'operazione lunga.

2. **Condivisione delle risorse:** i thread condividono automaticamente la memoria e le risorse del processo a cui appartengono, semplificando la comunicazione e l'accesso ai dati rispetto a processi separati.
3. **Economia (Overhead):** creare e gestire un thread, così come il cambio di contesto tra thread, è meno costoso (richiede meno tempo e risorse) rispetto alla creazione e gestione di un processo.
4. **Scalabilità:** sui sistemi multiprocessore/multicore, i thread possono essere eseguiti in parallelo su core distinti, massimizzando l'uso dell'hardware disponibili e incrementando il parallelismo.

Programmazione multicore

La progettazione dei sistemi è passata dai sistemi a singola cpu a quelli multicore o multiprocessore (dove ogni core appare come un processore separato al SO).

La **programmazione multithread** è il meccanismo chiave per utilizzare in modo efficiente questi sistemi e sfruttare al meglio la concorrenza.

In un sistema a singolo core, l'esecuzione concorrente si ottiene con l'avvicendamento nel tempo (interleaved) dei thread, dando l'illusione del parallelismo, ma eseguendo di fatto un solo thread alla volta. In un **sistema multicore**, l'**esecuzione parallela è effettiva**, poiché il sistema può assegnare thread diversi a core diversi per funzionare simultaneamente.

- Un sistema **concorrente** supporta più task che progrediscono nell'esecuzione.
- Un sistema **parallelo** esegue **simultaneamente** più task.

È possibile avere concorrenza senza parallelismo.

Legge di Amdahl

La legge di Amdahl è fondamentale per valutare il potenziale aumento delle prestazioni derivante dall'aggiunta di core.

- Determina il guadagno massimo di velocità (**Speedup**) per un'applicazione con una porzione seriale (S) su un sistema con N core:

$$\text{Speedup} \leq \frac{1}{S + \frac{(1-S)}{N}}$$

- La porzione seriale S di un'applicazione (la parte che non può essere eseguita in parallelo) ha un **effetto dominante** sulle prestazioni massime.
- Anche aumentando il numero di core all'infinito, l'incremento di velocità converge a $1/S$. Ad esempio, se il 50% di un'applicazione è seriale ($S = 0.5$), il massimo aumento di velocità possibile è di sole 2 volte, indipendentemente dal numero di core aggiunti.

Sfide della programmazione multicore

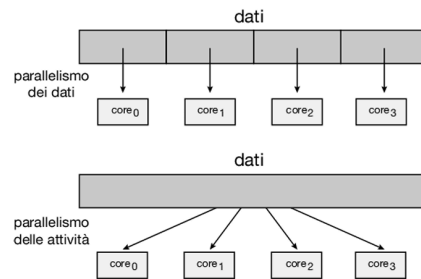
L'utilizzo ottimale dei sistemi multicore pone cinque sfide principali per i programmatori:

1. **Identificazione dei task:** esaminare l'applicazione per individuare aree separabili in task distinti ed eseguibili in parallelo.
2. **Bilanciamento (Load Balancing):** assicurarsi che i task paralleli abbiano una mole e un valore confrontabili per giustificare l'uso di core separati.
3. **Suddivisione dei dati:** dividere anche i dati a cui i task accedono in modo che possano essere manipolati efficientemente su core distinti.
4. **Dipendenze dei dati:** verificare se un task dipende dai dati prodotti da un altro. Se sì, è necessaria la sincronizzazione tra i task per soddisfare tali dipendenze.
5. **Test e debugging:** la natura parallela e i molteplici flussi di esecuzione rendono il testing e il debugging dei programmi più complessi rispetto alle applicazioni a thread singolo.

Tipi di parallelismo

Esistono due tipi di parallelismo:

- **Parallelismo dei dati:** riguarda la distribuzione di sottoinsiemi di dati su più core di elaborazione e l'esecuzione della stessa operazione su ogni core.
- **Parallelismo delle attività:** prevede la distribuzione di attività (thread), e non di dati, su più core.



Modelli di supporto al multithreading

I thread possono essere distinti in:

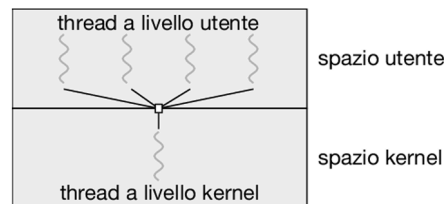
- **Thread a livello utente:** gestiti sopra il livello del kernel e senza il suo supporto.
- **Thread a livello kernel:** gestiti direttamente dal SO.

Deve esistere una relazione tra i thread a livello utente e i thread a livello kernel:

1. Modello molti a uno (many-to-one)

Mappa **molti thread a livello utente** su un **singolo thread a livello kernel**.

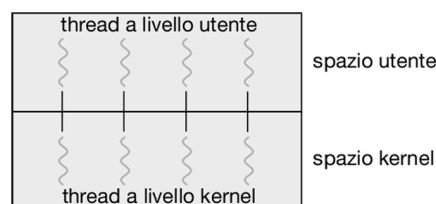
- **Vantaggi:** la gestione dei thread è molto efficiente perché avviene interamente nello spazio utente tramite una libreria di thread.
- **Svantaggi:**
 - **blocco:** se un thread utente esegue una chiamata di sistema bloccante, l'intero processo si blocca, impedendo a tutti gli altri thread utente di progredire.
 - **Nessun parallelismo:** è impossibile eseguire thread utente multipli in parallelo sui sistemi multicore, poiché solo un thread kernel può essere schedato alla volta.
- **Utilizzo:** usato raramente a causa dell'incapacità di sfruttare i multicore.



2. Modello da uno a uno (one-to-one)

Questo modello mappa **ogni thread a livello utente** su un **thread a livello kernel separato**.

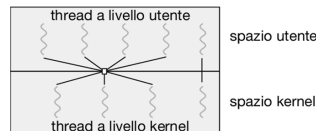
- **Vantaggi:**
 - **concorrenza/parallelismo:** offre un alto grado di concorrenza. Permette l'esecuzione parallela dei thread sui sistemi multiprocessore;
 - **nessun blocco totale:** se un thread utente esegue una chiamata bloccante, il kernel può schedare un altro thread kernel corrispondente a un altro thread utente, mantenendo l'applicazione attiva.
- **Svantaggi:**
 - **Overhead:** la creazione di ogni thread utente comporta il carico (overhead) della creazione di un thread kernel. Questo può sovraccaricare le prestazioni se si creano troppi thread.
- **Utilizzo:** è il più comune nei SO moderni come Linux e Windows.



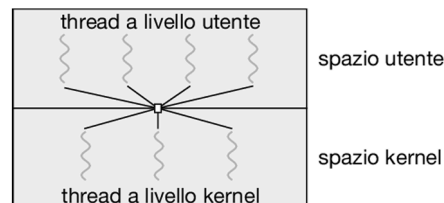
3. Modello da molti a molti (many-to-many)

Questo modello mappa **più thread a livello utente** su un numero **minore o uguale** di thread a livello kernel. Il numero di thread kernel può variare a seconda dell'applicazione o del sistema (es. più core = più thread kernel).

- **Vantaggi:**
 - **flessibilità:** combina il meglio dei due modelli precedenti.
 - I programmatori possono creare liberamente molti thread utente senza limitazioni.
 - Garantisce il parallelismo (i thread kernel possono eseguire i thread utente in parallelo).
 - Se un thread utente è bloccato, il kernel può assegnare un altro thread utente a un thread kernel disponibile.
- **Variante (Modello a due livelli):** una variante permette anche di vincolare alcuni thread utente specifici a singoli thread kernel.



- **Utilizzo:** sebbene sia il più flessibile, è difficile da implementare e, dato l'aumento dei core, la maggior parte dei SO predilige oggi la semplicità e la flessibilità (ormai sufficiente) del modello da uno a uno.



Librerie dei Thread

Una libreria dei thread fornisce al programmatore un'API (interfaccia di programmazione dell'applicazione) per creare e gestire i thread. L'implementazione di queste librerie può avvenire secondo due modalità principali:

Tipo di Libreria	Posizione	Invocazione Funzione	Note
A livello Utente	Interamente nello spazio utente , senza ricorso al kernel.	Chiamata locale a una funzione nello spazio utente.	Codice e strutture dati risiedono nello spazio utente.
A livello Kernel	Supportata direttamente dal sistema operativo (Kernel).	Generalmente, provoca una chiamata di sistema al kernel.	Codice e strutture dati risiedono nello spazio kernel.

Le tre librerie più usate sono:

- **Pthreads (POSIX):** estensione dello standard POSIX, può essere realizzata sia a livello utente che a livello kernel.
- **Windows:** una libreria a livello kernel specifica per i SO Windows.
- **Java:** l'API per i thread di Java è gestita dalla JVM (Java Virtual Machine), ma viene quasi sempre implementata utilizzando la libreria dei thread del SO (es. Pthreads, API Windows).

Condivisione dei dati:

- **Dati globali:** nelle librerie come Pthreads e Windows, i dati dichiarati a livello globale (al di fuori di ogni funzione) sono automaticamente condivisi tra tutti i thread dello stesso processo. In Java, l'accesso ai dati condivisi deve essere assegnato in modo esplicito.
- **Dati locali:** poiché ogni thread ha il suo proprio stack, i dati locali di una funzione (memorizzati nello stack) sono copie private per ciascun thread.

Gruppi di thread

Un server che crea un nuovo thread per ogni richiesta concorrente presenta due problemi:

1. **Latenza di creazione:** il tempo richiesto per creare il thread (e poi terminarlo) prima di poter iniziare e servire la richiesta è uno spreco di tempo.

2. **Esaurimento risorse:** permettere un numero illimitato di thread può esaurire le risorse del sistema (memoria, tempo di CPU), causando instabilità.

Il gruppo di thread è un'architettura che prevede la creazione di un numero predefinito di thread all'avvio del processo, che vengono mantenuti in un *pool* (gruppo) in attesa di lavoro.

1. **Creazione:** viene creato un numero fisso di thread all'inizio.
2. **Attesa:** i thread restano inattivi nel gruppo in attesa di una richiesta.
3. **Esecuzione:** quando il server riceve una richiesta, attiva un thread disponibile dal gruppo e gli passa il lavoro.
4. **Ritorno:** dopo aver completato il compito, il thread ritorna immediatamente nel gruppo per attendere la richiesta successiva, senza essere terminato.

Vantaggi:

- **velocità:** servire una richiesta tramite un thread già esistente è più rapido rispetto all'attesa della creazione di un nuovo thread;
- **limiti di risorse:** il gruppo limita il numero di thread concorrenti, impedendo l'esaurimento delle risorse di sistema;
- **flessibilità di esecuzione:** consente di separare il compito dalla sua creazione, permettendo di utilizzare strategie di scheduling diverse (es. esecuzione periodica o ritardata)

Fork-Join

Il modello Fork-Join è una strategia di threading sincrona che definisce un preciso flusso di esecuzione tra i thread padre e figlio:

1. **Fork (Creazione):** il thread padre crea (o "forka") uno o più thread figli per distribuire il lavoro.
2. **Attesa:** il thread padre attende che tutti i thread figli completino il loro lavoro.
3. **Join (Unione):** i thread figli, una volta terminata l'esecuzione, si uniscono ("joinano") al padre.
4. **Combinazione:** solo dopo che tutti i figli si sono uniti, il padre può riprendere la propria esecuzione e recuperare e combinare i risultati ottenuti dai figli.

Il modello Fork-Join può essere utilizzato in due contesti:

1. **Creazione esplicita di thread**

È l'uso tradizionale, dove il programmatore specifica manualmente la creazione (fork) e l'attesa (join) di ogni thread.

2. **Threading implicito**

Questa è l'applicazione più moderna del modello.

- **Definizione di task:** il programmatore non crea direttamente i thread, ma definisce i task paralleli che devono essere eseguiti.
- **Gestione della libreria:** una libreria si occupa della gestione e della creazione effettiva dei thread e dell'assegnazione dei task ad essi.
- **Thread pool sincrono:** il modello fork-join implicito è paragonabile a una versione sincrona di un thread pool, dove la libreria (invece del programmatore) decide il numero ottimale di thread da creare e utilizzare (spesso basandosi su euristiche come il numero di core del sistema).

Chiamate di sistema fork() ed exec()

Nei programmi multithread, la creazione di un nuovo processo (`fork()`) e la sostituzione dell'immagine del processo (`exec()`) hanno effetti specifici sui thread.

`fork()` (Creazione di processo)

Quando un thread invoca `fork()`, il nuovo processo figlio può nascere in due modi:

1. **Duplicato di tutti i thread:** il processo figlio eredita una copia di tutti i thread del padre.
2. **Duplicato del solo thread invocante:** il processo figlio contiene solo il thread che ha chiamato `fork()`.

La scelta dipende dall'uso:

- Se `exec()` viene chiamata immediatamente dopo `fork()`, è più efficiente duplicare solo il thread invocante, poiché il nuovo programma (`exec()`) sostituirà comunque l'intero processo e tutti i suoi thread.
- Se `exec()` NON segue immediatamente `fork()`, il nuovo processo figlio necessita di una copia di tutti i thread per continuare le attività parallele del padre.

`exec()` (Sostituzione di immagine)

Se un thread invoca `exec()`, il nuovo programma specificato sostituisce l'intero processo esistente, inclusi tutti i suoi thread.

Gestione dei segnali

I segnali vengono usati nei sistemi Unix per notificare eventi ai processi. Nei processi multithread, il problema è decidere a quale thread il segnale debba essere inviato.

Tipi di segnali:

- **Sincroni:** generati da un'azione all'interno del processo (es. divisione per zero). Vengono inviati al thread che ha causato l'evento.
- **Asincroni:** generati da eventi esterni (es. un segnale da tastiera [`Ctrl+C`]). La destinazione è meno chiara.

Destinazione del segnale in un processo multithread

Le opzioni di recapito sono:

1. Al thread che ha causato l'evento (obbligatorio per i segnali sincroni).
2. A tutti i thread del processo.
3. A specifici thread.
4. A un thread dedicato alla ricezione di tutti i segnali (thread gestore).

Nella maggior parte delle implementazioni Unix (es. Pthreads), un segnale asincrono viene inviato solo al primo thread che non lo blocca (i thread possono specificare quali segnali accettare).

`pthread_kill(pthread_t id, int signal)` è la funzione che permette di inviare un segnale a un thread specifico.

Cancellazione dei thread

La cancellazione dei thread è l'operazione che permette di terminare un thread bersaglio (target thread) prima che completi il suo lavoro (es. annullare un download in un browser).

Esistono due modalità principali, con implicazioni critiche sulla gestione delle risorse:

Modalità	Funzionamento	Problemi/Rischi	Vantaggi
Asincrona	Il thread bersaglio viene terminato immediatamente .	Alto rischio di perdita di risorse o corruzione di dati condivisi (se terminato nel mezzo di un aggiornamento).	Terminazione immediata.
Differita	Il thread bersaglio controlla periodicamente (in un <i>punto di cancellazione</i>) se deve terminare.	L'effettiva terminazione è ritardata.	Permette di terminare in modo ordinato , rilasciando le risorse (gestore della pulizia).

Pthreads sconsiglia la cancellazione asincrona e usa la differita come predefinita. La terminazione avviene solo quando il thread raggiunge un punto di cancellazione (es. durante chiamate di sistema bloccanti o tramite la funzione `pthread_testcancel()`).

Dati locali dei thread (TLS) e Attivazione dello Scheduler

Sebbene i thread condividano la memoria, a volte è necessario che ogni thread abbia una copia privata di certi dati per l'intera durata della sua vita (visibile attraverso tutte le chiamate di funzione, a differenza delle variabili locali dello stack).

- **Scopo:** usato per associare un identificatore unico (es. ID di transazione) a ciascun thread.
- **Implementazione:** simile ai dati statici, ma unica per ogni thread. Librerie e linguaggi (es. Java con `ThreadLocal`, Pthreads con `pthread_key_t`, ecc.) forniscono strumenti per definirli.

Nei modelli di threading più complessi (Molti a molti o a Due Livelli), è necessario coordinare la libreria utente e il kernel per una gestione dinamica dei thread.

- **LWP (Lightweight Process):** è una struttura dati intermedia tra i thread utente e i thread kernel. Appare alla libreria utente come un processore virtuale su cui schedulare i thread utente. Ogni LWP è legato a un thread kernel che esegue il lavoro sul processore fisico.

- **Attivazione dello scheduler:** è un modello di comunicazione in cui il kernel fornisce gli LWP all'applicazione. Il kernel utilizza un meccanismo chiamato upcall per informare l'applicazione (tramite un gestore nella libreria dei thread) quando un evento critico sta per verificarsi (es. quando un thread sta per bloccarsi). Questo permette alla libreria di salvare lo stato del thread e di reschedulare un altro thread su un LWP disponibile.

Thread di Linux

Linux non distingue tra processi e thread, impiegando generalmente al loro posto il termine **task** in riferimento a un flusso del controllo nell'ambito del programma. Quando `clone()` è invocata, riceve come parametro un insieme di indicatori (flag), al fine di stabilire quante e quali risorse del task genitore debbano essere condivise dal task figlio. Adoperare `clone()` in questo modo è equivalente a creare thread come descritto precedentemente, dal momento che il task genitore condivide la maggior parte delle proprie risorse con il task figlio.